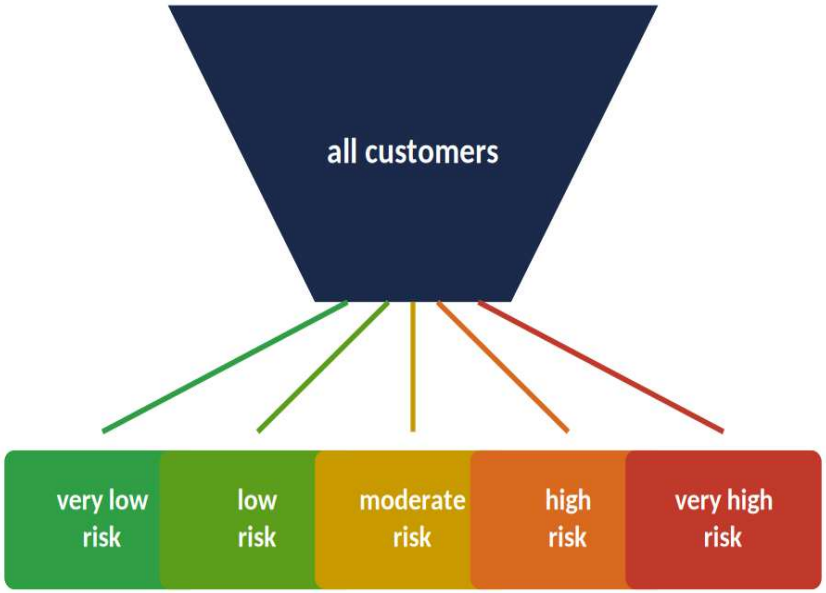
Credit Repayment Risk Profiling Engine

Team members:

Apoorva Rampal Chadda – V01054205
Priyansh Patel –
Maria Malini –
Nikhil Kumar –
Soorya Vasudevan –

What We're Building



This project builds the modeling core of a customer risk profiling engine that a bank could plug into its credit approval process: instead of a binary accept/reject call, it estimates each customer's likelihood of falling behind on their credit card payment next month and translates that likelihood into a small set of risk tiers a credit team can act on directly.

Goals & Objectives

1

Goal 1

Build ML model to predict the likelihood of a customer to fail on next credit payment.

2

Goal 2

Strategies to resolve Class Imbalance

3

Goal 3

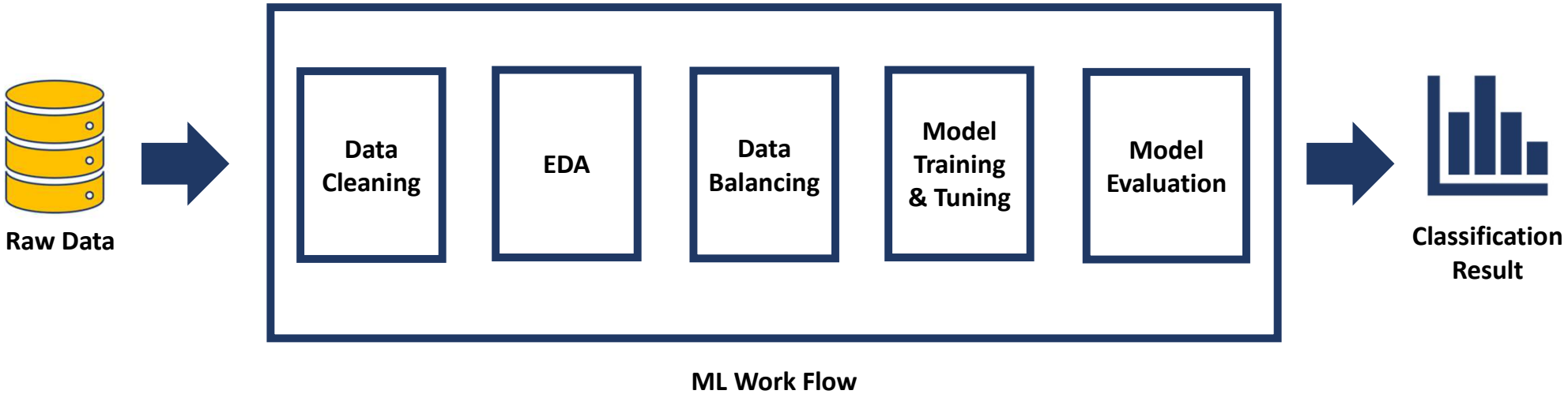
Data leakage risk - Resampling and imputation had to be fit only on train data, applied to a sealed test set

4

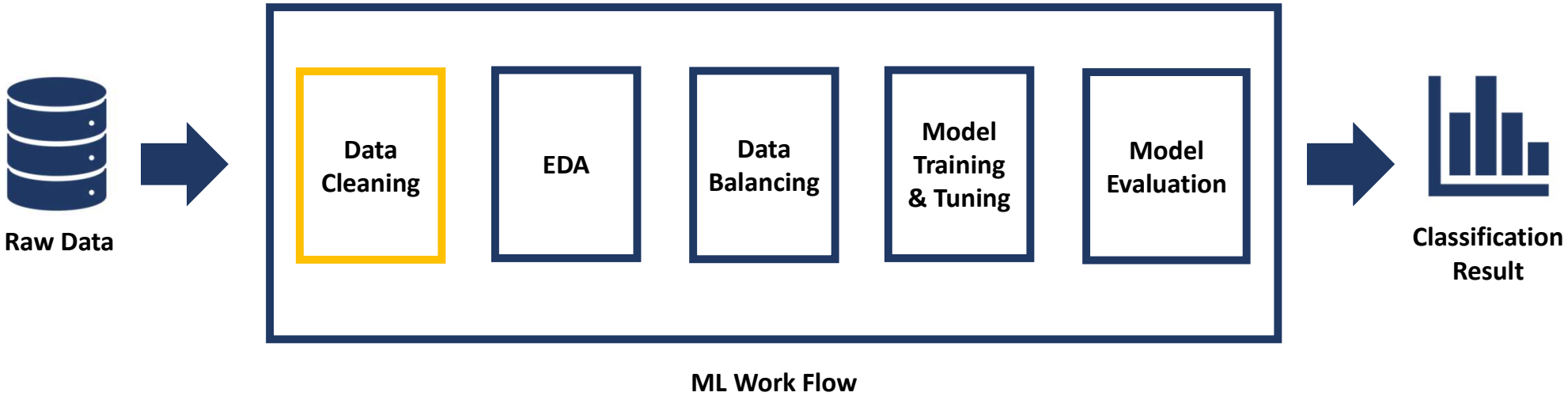
Goal 4

Develop Strategies to compare and evaluate ML models

Project Architecture



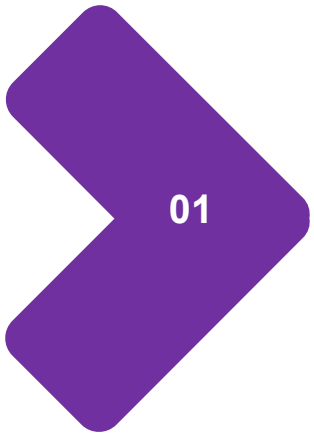
Data Cleaning



Data Cleaning: Dealing with Missing Values

Missing Categorical Variables

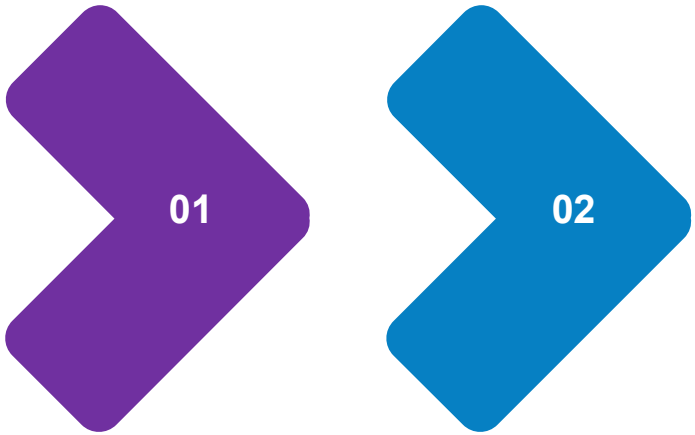
Stored the Missing values in a new category called “UNKNOWN”



Data Cleaning: Dealing with Missing Values

Missing Categorical Variables

Stored the Missing values in a new category called “UNKNOWN”



Missing Numerical Variables

700 values (or 2.9%) missing, Missing values replaced with column Mean.

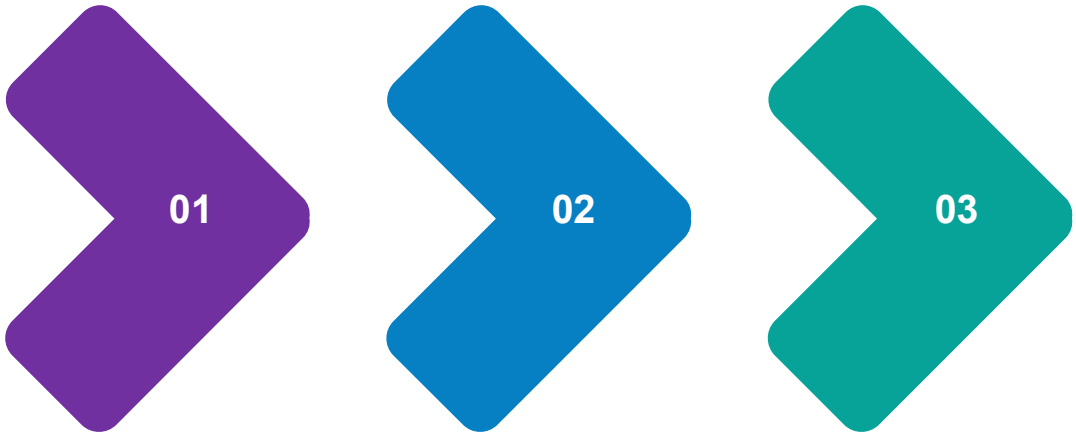
Data Cleaning: Dealing with Missing Values

Missing Categorical Variables

Stored the Missing values in a new category called “UNKNOWN”

Dropping Unnecessary Columns

WORK_YEARS – 85% Values Missing
ID – Represents client ID



Missing Numerical Variables

700 values (or 2.9%) missing, Missing values replaced with column Mean.

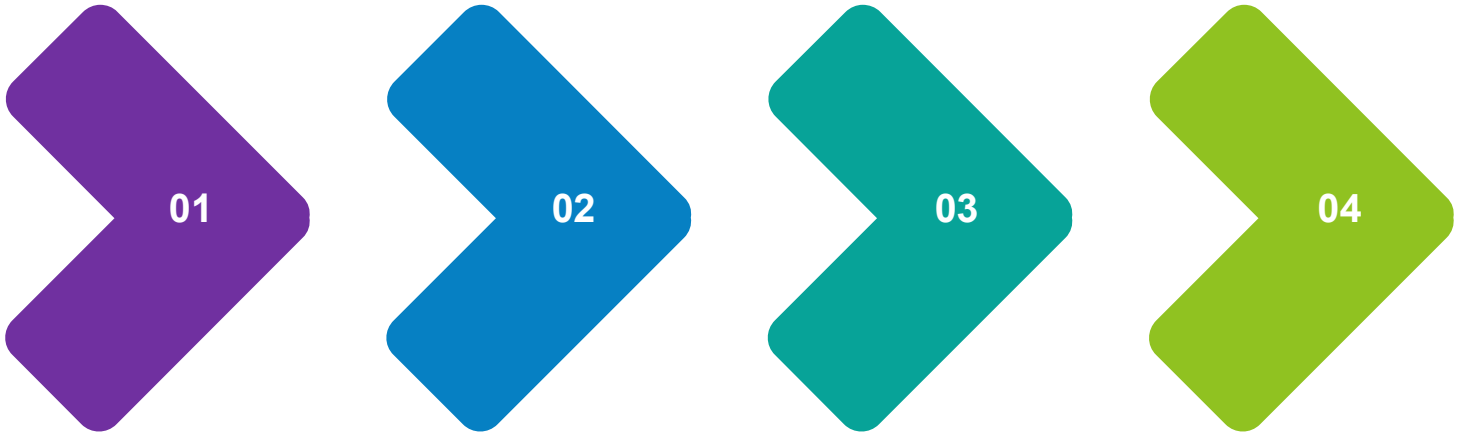
Data Cleaning: One-Hot Encoding

Missing Categorical Variables

Stored the Missing values in a new category called “UNKNOWN”

Dropping Unnecessary Columns

WORK_YEARS – 85% Values Missing
ID – Represents client ID



Missing Numerical Variables

700 values (or 2.9%) missing, Missing values replaced with column Mean.

Encoding Categorical Variables

One-hot encoding categorical variables into binary vectors of 1 & 0

Data Cleaning: Feature Scaling

Missing Categorical Variables

Stored the Missing values in a new category called “UNKNOWN”

Dropping Unnecessary Columns

WORK_YEARS – 85% Values Missing
ID – Represents client ID

Feature Scaling

Using StandardScaler to re-scale numerical features

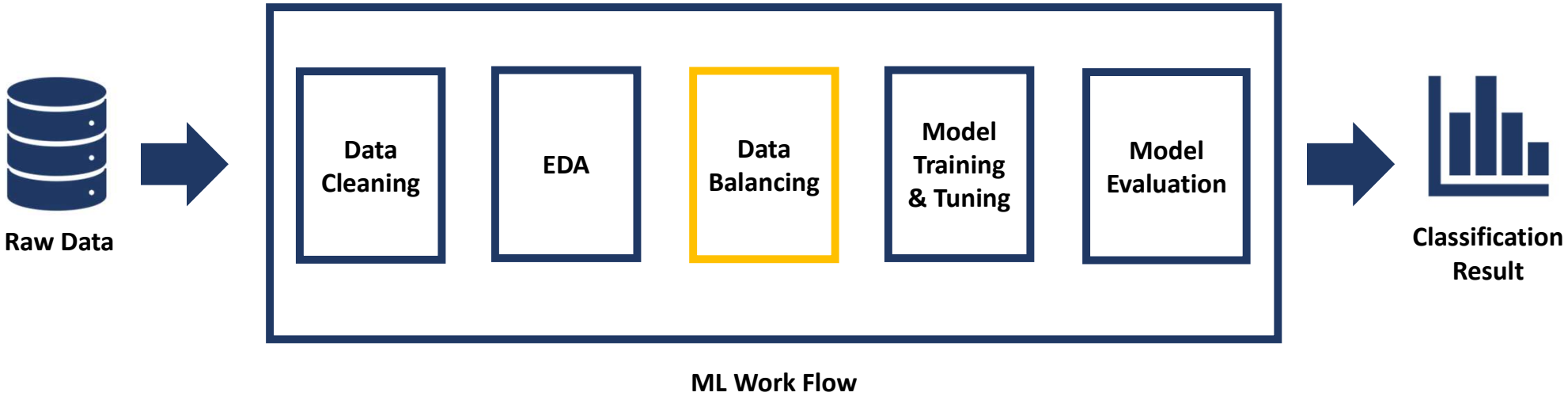
Missing Numerical Variables

700 values (or 2.9%) missing, Missing values replaced with column Mean.

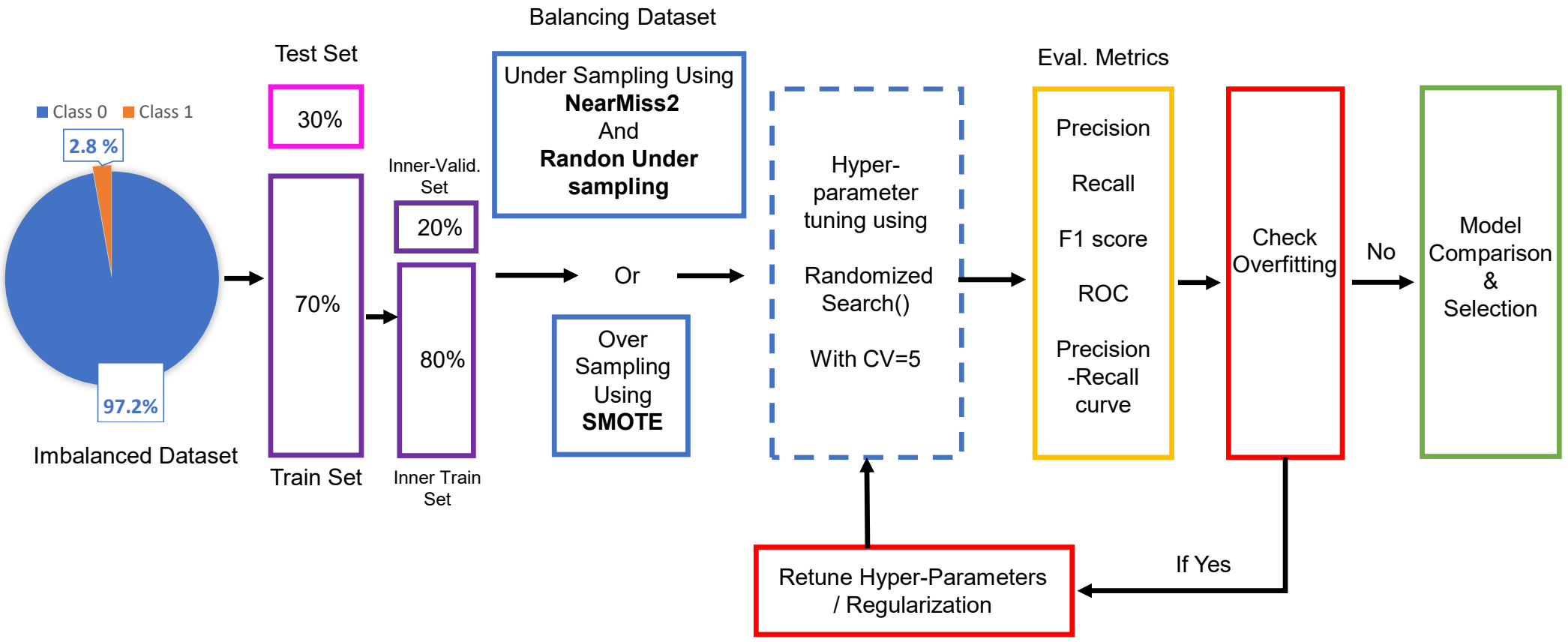
Encoding Categorical Variables

One-hot encoding categorical variables into binary vectors of 1 & 0

Data Balancing Strategies



ML Work Flow


Balancing Dataset
Test Set
Eval. Metrics
Class 0
Class 1
2.8 %
97.2%
Imbalanced Dataset
30%
70%
Train Set
Inner-Valid. Set
20%
80%
Inner Train Set
Under Sampling Using NearMiss2 And Randon Under sampling
Or
Over Sampling Using SMOTE
Hyper-parameter tuning using
Randomized Search()
With CV=5
Precision
Recall
F1 score
ROC
Precision -Recall curve
Check Overfitting
No
Model Comparison & Selection
If Yes
Retune Hyper-Parameters / Regularization

Building Models

Handling Class Imbalance

3 resampling techniques used to deal with class imbalance issue:

Random Under-sampling

Advantage
Simple, fast, most consistent results

Disadvantage
Discards majority-class data

NearMiss-2 Under-sampling

Advantage
In theory retains harder boundary cases

Disadvantage
Distance-based; dominated by unscaled large-value columns - ruled out

SMOTE Oversampling

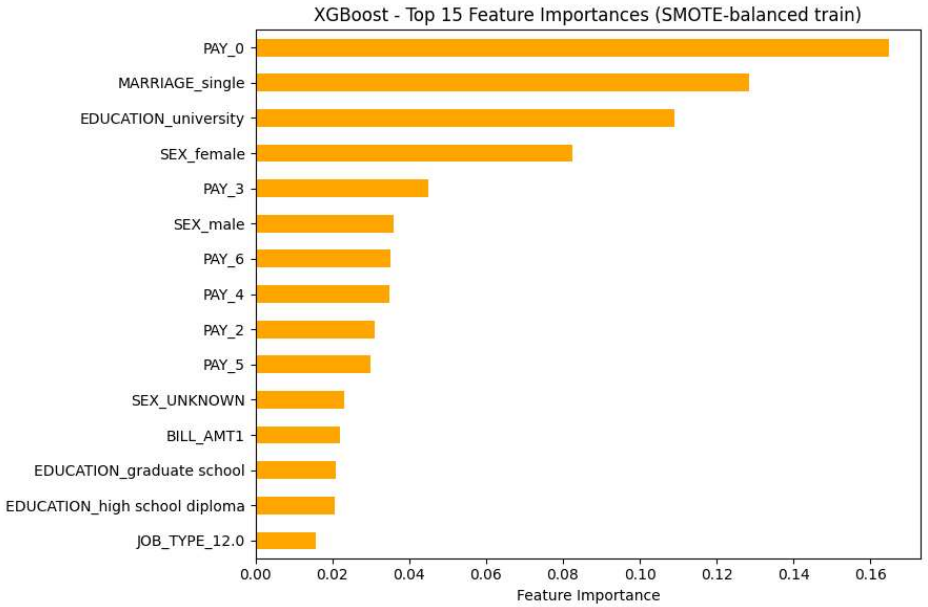
Advantage
Best ranking ability, keeps all real data

Disadvantage
Needed ImbPipeline to fix per-fold leakage

- Feature selection (importance + SHAP) narrowed 49 encoded features down to a final 14
- Hyperparameters tuned via RandomizedSearchCV with 5-fold cross-validation

Feature Selection (importance + SHAP)

- Not every one of the ~49 encoded features earns its place in the final model
- Built-in feature importance - quick, model-native ranking of what the trees split on
- SHAP analysis - shows what drives individual predictions
- Narrowed from 49 candidates to a final 14 features
- Dominated by PAY_0–PAY_6, LIMIT_BAL, BILL_AMT1, and a few demographic encodings



Selecting Best Model and Model Evaluation

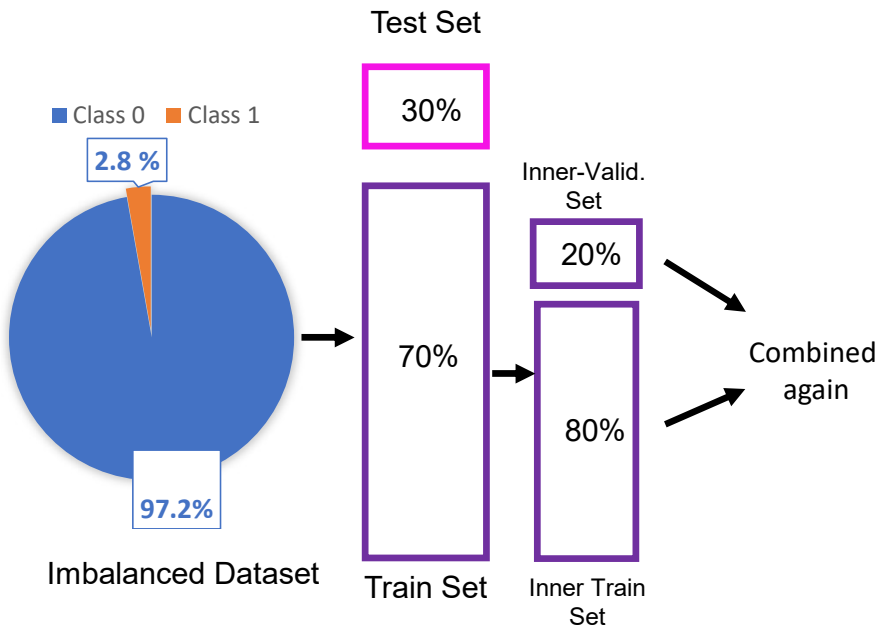
Nine models compared - Random Forest, XGBoost, LightGBM × 3 resampling techniques

Resampling	Model	Precision	Recall	F1	Accuracy
Random Undersampling	LightGBM	0.986	0.772	0.866	0.768
NearMiss-2	LightGBM	0.967	0.125	0.221	0.145
SMOTE	XGBoost	0.973	0.994	0.984	0.968

Best overall model: XGBoost on SMOTE. SMOTE keeps every real record intact rather than discarding data; paired with XGBoost - a proven algorithm for structured, tabular data - the combination carried forward into fine-tuning and final blind-test evaluation.

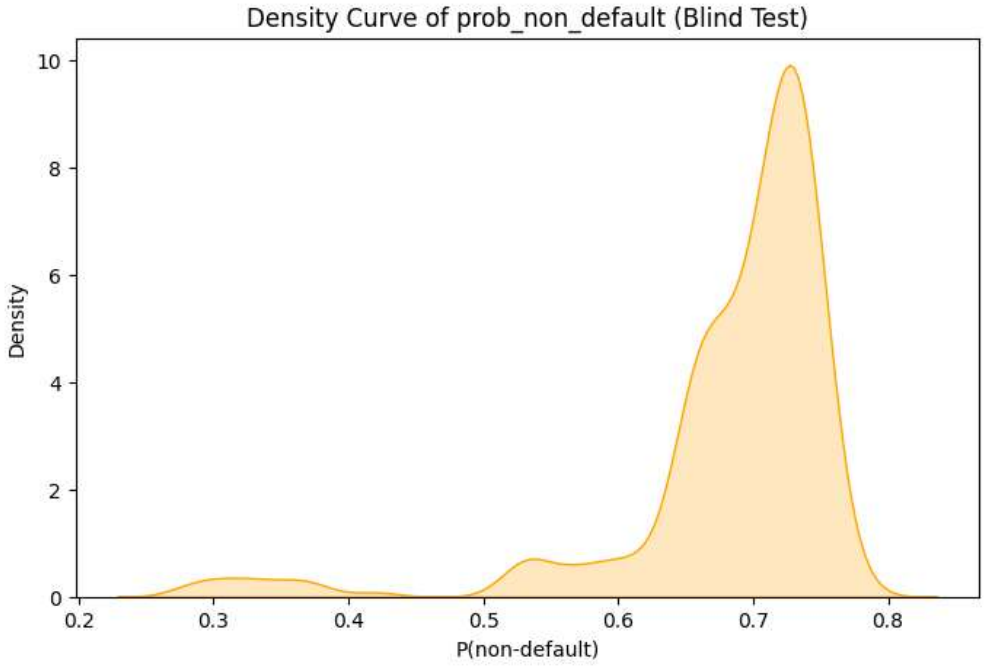
Fine-Tuning the Best Model

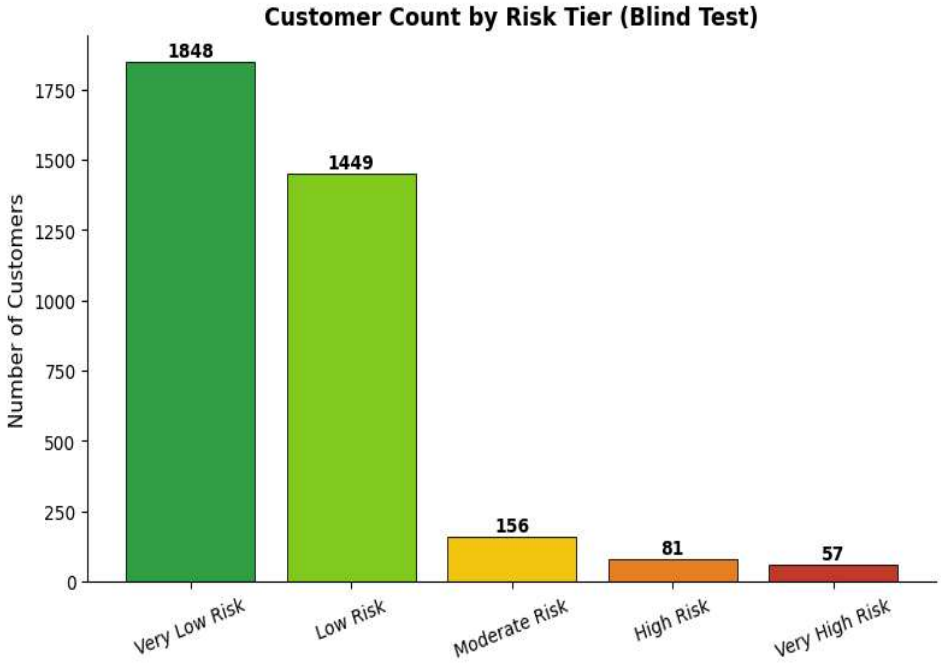
- Inner-train and inner-validation merged back together - trains on the full available data pool
- Only the 14 selected features (via feature importance & SHAP) were kept
- Hyperparameters re-tuned on this combined dataset via RandomizedSearchCV
- SMOTE + XGBoost wrapped in an ImbPipeline, re-tuned including scale_pos_weight



Customer Risk Profiling

Translating each customer's repayment likelihood into five actionable risk tiers

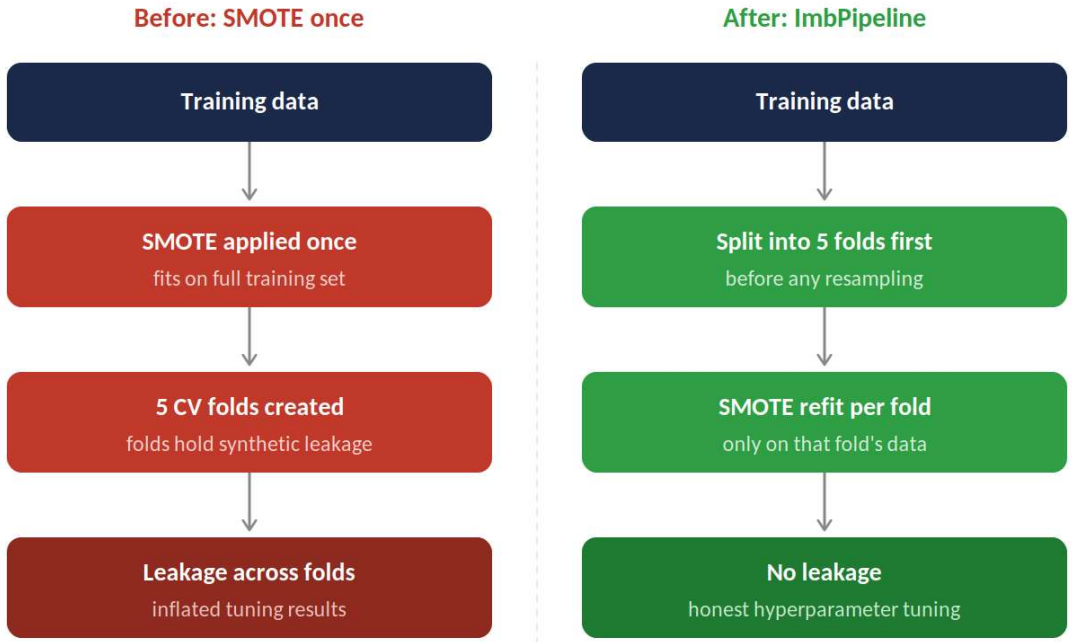




Most customers cluster in lower-risk tiers (1,848 Very Low Risk; 1,449 Low Risk), consistent with the ~97% non-default base rate - giving the credit team a workable segmentation to focus scrutiny where it matters.

Addressing Data Leakage

- Missing value imputation: mean imputation fit only on the training set, applied to both
- Resampling on train only: test set sealed and set aside before any balancing was applied
- Leakage inside hyperparameter tuning: SMOTE applied once before CV meant folds contained synthetic samples influenced by held-out data
- Fix - SMOTE per fold via ImbPipeline: refits SMOTE separately inside each CV fold, closing the leakage gap at the cross-validation level



Conclusions

1

XGBoost on SMOTE selected as the best overall model

keeps the full customer base while pairing with a proven tabular-data algorithm

2

Leakage-free pipeline built and verified end-to-end

from imputation through resampling to per-fold SMOTE in hyperparameter tuning

3

Customer risk profiling delivers actionable tiers

Use predict_proba() function to get the class probabilities and then translating raw probabilities into 5 risk tiers, a credit team can act on directly

4

Oversampling outperformed undersampling overall

Models trained on Over-sampled dataset give better perfomance than Models trained on under Under-sampled dataset